

Week 8: 0/1 Knapsack Problem

411025020

Questions

1. Analyze space and time complexity of a recursive implementation without cache.

```
vector<pair<int,int>> products;  
int Price(int n,int w)  
{  
    if(n<=0 || w<=0)  
        return 0;  
    if (products[n].second > w)  
        return Price(n - 1, w);  
    return max(Price(n-1,w),Price(n-1,w-  
products[n].second)+products[n].first);  
}
```

每次都需遞迴調用自身2次，且調用深度為 n 。

時間複雜度為 $O(2^n)$

空間複雜度為 $O(n)$

2. Design a table to cache the answer of subproblems.

```
vector<pair<int,int>> products;  
long Price(int n,int w)  
{  
    vector<long> dp(w+1,0);  
    for(int i=0;i<n;i++)  
    {  
        for(int j=w;j>=products[i].second;j--)  
            dp[j]=max(dp[j],dp[j-products[i].second]+products[i].first);  
    }  
    return dp[w];  
}
```

3. Analyze space and time complexity of implementation at Q2.

時間複雜度為 $O(n \times 2^{\log_2 w})$

空間複雜度為 $O(w)$

4. Please explain why the algorithm is a pseudo polynomial time algorithm (kind of exponential time algorithm), not a polynomial time algorithm.

因為此演算法有一個與資料數量無關的係數 w ，因此是偽多項式時間演算法，而不是多項式時間演算法。 $bit = \log_2 w$ ，時間複雜度為 $O(n \times 2^{bit})$

5. Solve <http://oj.csie.ndhu.edu.tw/problem/ALG05>

C++

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    int t,n,max_w,p,w,member,anser;
    long mx;
    cin>>t;
    while(t--)
    {
        vector<pair<int,int>> products;
        cin>>n;
        for(int i=0;i<n;i++)
        {
            cin>>p>>w;
            products.push_back(make_pair(p,w));
        }
        cin>>member;
        anser=0;
        while(member--)
        {
            mx=0;
            cin>>max_w;
            vector<long> dp(max_w+1,0);
            for(int i=0;i<n;i++)
            {
                for(int j=max_w;j>=products[i].second;j--)
                {
                    dp[j]=max(dp[j],dp[j-products[i].second]+products[i].first);
                }
            }
            mx=dp[max_w];
            anser+=mx;
        }
        cout<<anser<<endl;
    }
}
```